

Actividad 2 - CC4303 Redes

Link del repositorio en GitHub: [Repositorio](#)

En esta actividad no se utilizó la inteligencia artificial.

Integrantes:

- **Oscar Conejeros Bruce**
- **Julio Yáñez**

Sobre la actividad

En esta actividad, se pidió construir un resolver DNS, el cual tenga una memoria caché. Para esto, asumiremos que todas las consultas que se harán del cliente al resolver serán de tipo A, es decir, preguntarán por la dirección IPv4 del dominio consultado.

Prerrequisitos

Para esta actividad, el único prerrequisito es tener la librería dnslib de Python. Esta se instala con el siguiente código:

```
pip3 install dnslib
```

O desde el repositorio de GitHub de dnslib: <https://github.com/paulc/dnslib>

Cómo ejecutar el servidor DNS

Para ejecutar en el servidor, hay que ejecutar el programa python **resolver.py**, que se encuentra en la misma carpeta actividad_2.

Para ejecutar el programa en modo debug basta con ejecutar el programa **resolver.py** con el argumento -d (o --debug), de la siguiente manera:

```
python3 resolver.py -d
```

Luego, para hacer consultas al servidor, se usa el comando dig en la terminal de la siguiente manera:

```
dig -p8000 @[IP_SERVIDOR_DNS] [dominio_a_consultar]
```

Decisiones creativas

En esta actividad, tal como se mencionó en la materia de DNS, se usaron sockets no orientados a conexión, para asegurar una respuesta lo más rápida posible al servidor DNS.

Además, como se optó por utilizar la librería `dnslib`, se dio uso al parser ya incluido por esta librería, el cual otorgaba una estructura de datos ya adecuada para el manejo correcto de los mensajes DNS.

Por último, en la memoria caché, se decidió usar como estructura de datos un diccionario, el cual tiene como llaves los dominios consultados y como valores la lista de Resource Records de la respuesta, para entregar un mensaje DNS con este listado más fácilmente en una respuesta futura. Ver función **`gen_new_cache(last)`**, ya que esta es la que se encarga de crear la memoria caché.

El resto del programa sigue las instrucciones dadas en el enunciado de la actividad.

Es importante recalcar que una limitación de ignorar cualquier otro tipo de respuestas no consideradas en el paso 4 es, por ejemplo, que no podemos leer registros de tipo AAAA, es decir, no podremos obtener direcciones IP de 128 bits.

Implementación

El programa consiste en 3 funciones, **`gen_new_cache(last: List)`**, **`resolver(mensaje_consulta: bytes, ip_addr="198.41.0.4": str)`** y el programa principal, que se ejecuta automáticamente.

- **`gen_new_cache(last: List)`**: Esta función recibe una lista con tuplas correspondientes al dominio consultado y la sección de respuesta del mensaje DNS recibido. Se encarga de generar una memoria caché para el resolver y actualizarla luego de cada consulta al servidor.
- **`resolver(mensaje_consulta: bytes, ip_addr="198.41.0.4": str)`**: La función sigue los pasos detallados en la parte 4 del enunciado de la actividad y finalmente, si no se cumple ninguna de las condiciones indicadas en los pasos, retorna el último mensaje recibido, junto a un mensaje indicando que la respuesta no es soportada por el resolver.
- **Programa principal**: Este es el que maneja toda la lógica, recibe los mensajes del cliente, llama a la función resolver, entrega la respuesta al cliente y actualiza el listado de las últimas consultas y la memoria caché.

Resultados de la experimentación

Al momento de realizar la consulta:

```
dig -p8000 @[IP_SERVIDOR_DNS] eol.uchile.cl
```

se recibieron 12 respuestas, una de tipo CNAME con rdata `oeol-c.uchile.cl` y todas las demás con una IP de la forma `146.83.63.XX`.

Luego, realizando la misma consulta, se logra obtener las mismas respuestas, pero esta vez desde la memoria caché (mensaje debug: (debug) se utilizó el caché).

El comando:

```
dig -p8000 @[IP_SERVIDOR_DNS] www.uchile.cl
```

resuelve a 200.89.76.36, mientras que el comando:

```
dig -p8000 @[IP_SERVIDOR_DNS] cc4303.bachmann.cl
```

resuelve a 104.248.65.245.

Siguiendo con los experimentos de la actividad, podemos ver que al realizar la consulta:

```
dig -p8000 @[IP_SERVIDOR_DNS] www.webofscience.com
```

se resuelve correctamente, obteniendo 4 respuestas: 2 de tipo CNAME y 2 de tipo A. Las respuestas de tipo CNAME tienen un rdata `www-us.webofscience.com.` y `www-us.webofscience.com.cdn.cloudflare.net.` respectivamente, y las de tipo A entregan las IP `104.18.25.24` y `104.18.24.24`, por lo que no hay ningún problema que se deba resolver en este caso.

Para el comando:

```
dig -p8000 @[IP_SERVIDOR_DNS] www.cc4303.bachmann.cl
```

no se obtiene una respuesta, ya que la respuesta que recibe el servidor para resolver no contiene ningún resource record de tipo A en la respuesta, ni recibe un resource record de tipo NS en la sección authority, por lo que, según enunciado, se debe ignorar, provocando un connection timed out para el cliente.

En este caso se esperaba una respuesta de parte del servidor equivalente al comando ejecutado con `cc4303.bachmann.cl`, pero luego de realizar la misma consulta a `@1.1.1.1`, se puede apreciar que esta recibe respuesta de tipo SOA en la sección authority, con un rname `bachmann.cl.` y rdata `ns1.digitalocean.com. hostmaster.bachmann.cl. 0 10800 3600 604800 1800`. Esto probablemente ocurre porque se está dando un name server donde podría encontrarse `www.cc4303.bachmann.cl`, ya que no se logró encontrar esta dirección.

Para el último experimento se hicieron 10 consultas de la siguiente forma:

```
dig -p8000 @[IP_SERVIDOR_DNS] example.com
```

donde todas fueron en distintas instancias del servidor (se reiniciaba luego de cada consulta) con el fin de no utilizar la memoria caché. Finalmente, como resultado se obtuvo que en las 10 consultas, siempre se hicieron al mismo name server ("`.`", con IP `198.41.0.4`). Esto seguramente se debe a que nuestro resolver es de tipo recursivo.